

Module 1: Introduction to Programming & Basic Java Syntax

Sarah Foss

Objectives

To explain what a program and programming is

To create, compile, and run your first Java program

To explain the basic syntax of the Java language

To understand conventions, code style, and documentation

To distinguish between different types of errors

Algorithm

An ***algorithm*** is a precise and systematic method for solving a problem.



Programming

What is programming?

Programming is the process of constructing programs in order to instruct a computer on how to solve problems. It is the act of writing out the steps of an *algorithm*.

A **program** is a sequence of simple computer instructions in some *language* which tell the computer the necessary steps to solve a problem or complete a task.

A **language** is the structure and syntax used to communicate to the computer the tasks it is required to perform.

We all "program" by giving instructions to others!

Machine Language

Machine language, also known as machine code, is the most basic programming language that computers understand.

It consists of binary code (combinations of 0s and 1s) that directly instruct the computer's hardware to perform specific tasks, like moving data or performing calculations. Since it communicates directly with the computer's central processing unit (CPU), machine language is very efficient but also difficult for humans to write, read, or modify.

For example, to add two numbers, you might write an instruction in binary like this:

1101101010011010

Assembly Language

Assembly language is a low-level programming language that uses simple symbolic names to represent machine code instructions.

It's easier to read and write than machine language but still closely tied to the computer's hardware. An assembler translates assembly code into machine language, making it usable by the computer's CPU.

For example, here is a simple snippet of assembly code that demonstrates adding the contents of two registers and then storing the result in a third register:

```
mov  eax, 5
mov  ebx, 3
add  eax, ebx
mov  ecx, eax
```

High- Level Languages

A high-level language is a programming language designed to be easy for humans to read and write, using clear instructions similar to everyday language. It allows programmers to write complex tasks without worrying about the computer's hardware details.

For example, the following is a set of high-level language statements that computes the area of a rectangle:

```
length = 5;  
width = 3;  
area = length * width;
```


Examples of High-Level Languages

C

C++

C#

Java

JavaScript

Python

PHP

Translating Source Code

Computers translate high-level **source code** into machine language through two primary methods:

Compilation:

A program called a compiler translates the entire high-level code into **machine code** before the program runs.

Interpretation:

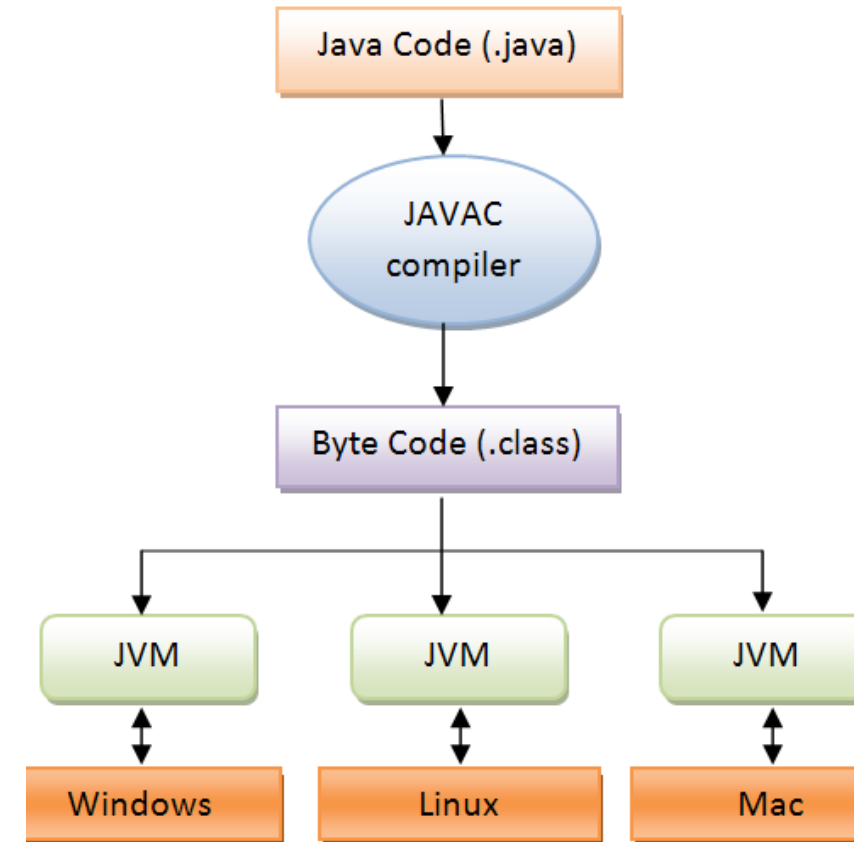
An interpreter translates high-level code into machine language in a line-by-line manner as the program is run. This means that no machine code file is produced beforehand.

Java: A Hybrid Approach

With most compiled languages, you must recompile the source program when it is ported to another machine because the compiled program is tailored to run on a specific machine only.

Java, on the other hand, is designed to make programs run on any platform. With Java, you write your program once and compile it into a special form of object code called **bytecode**.

This bytecode can run on any computer that has a Java Virtual Machine (JVM), which is software that interprets the Java bytecode.



Your First Java Program

```
// This program prints Hello World!

public class FirstProgram {
    public static void main(String[] args) {
        System.out.println("Hello World!");
    }
}
```

Create/Modify Source Code
FirstProgram.java

```
root@OC-507875: ~
GNU nano 6.2 FirstProgram.java *
public class FirstProgram{
    public static void main(String[] args){
        System.out.println("Hello World!");
    }
}
```

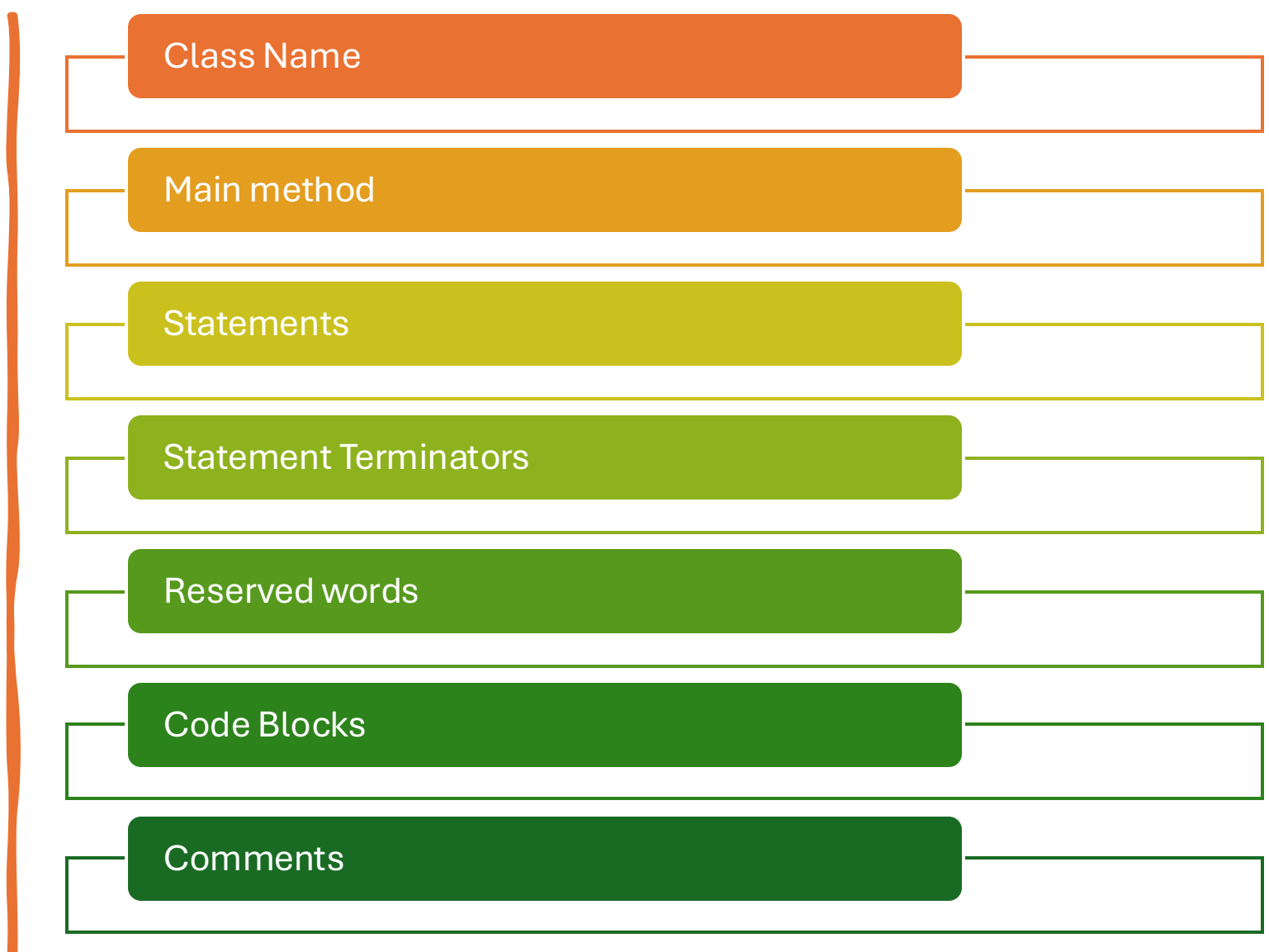
Compile Source Code into Bytecode
javac FirstProgram.java

If compilation error(s)

```
root@OC-507875: ~
GNU nano 6.2 FirstProgram.class
?????^@^@7^@^]
^@^F^@O      ^@^P^@Q^H^@R
^@^S^@T^G^@U^G^@V^A^@F<init>^A^@C()V^A^@DCode^A^@OLineNumberTable^A^@Dmai>
SourceFile^A^@QFirstProgram.java^L^@G^@H^G^@W^L^@X^@Y^A^@LHello World!^G^@>
^@^@^@F^@A^@^@^@A^@ ^@^K^@L^@A^@ ^@^@^@%^@B^@A^@^@^@ ^@^B^R^C^@^D^@>
^@^@^@
^@B^@^@^@C^@H^@D^@A^@M^@^@B^@N
[ Read 7 lines ]
```

Run Bytecode
java FirstProgram

Components of a Java Program



Class Name

Main method

Statements

Statement Terminators

Reserved words

Code Blocks

Comments

Class Name

In Java, every piece of code must be inside a class, and the class name should start with a capital letter.

For example, we called our class FirstProgram:

```
// This program prints Hello World!

public class FirstProgram {
    public static void main(String[] args) {
        System.out.println("Hello World!");
    }
}
```

Main Method

Every Java program must have a main method.

It serves as the entry point for starting up the application.

```
// This program prints Hello World!
```

```
public class FirstProgram {  
    public static void main(String[] args) {  
        System.out.println("Hello World!");  
    }  
}
```


Statements

In Java, a statement is a piece of code that performs a specific action.

Below, we use the `System.out.println()` method in a statement to print a line of text to the console.

```
// This program prints Hello World!

public class FirstProgram {
    public static void main(String[] args) {
        System.out.println("Hello World!");
    }
}
```

Statement Terminator

Each statement in Java ends with a **statement terminator**, a semicolon (;)

You can think of it like a period at the end of a sentence in English!

```
// This program prints Hello World!
```

```
public class FirstProgram {  
    public static void main(String[] args) {  
        System.out.println("Hello World!");  
    }  
}
```

Reserved Words

In Java, a reserved word (or keyword) is a special word that the Java language has set aside for a specific purpose.

These words have specific functions and cannot be used as names for variables, methods, or classes.

```
// This program prints Hello World!
```

```
public class FirstProgram {  
    public static void main(String[] args) {  
        System.out.println("Hello World!");  
    }  
}
```

Code Block

The curly braces {} marks the beginning and the end of a block of code.

It defines a space where certain code statements can be executed together.

```
// This program prints Hello World!
```

```
public class FirstProgram {  
    public static void main(String[] args) {  
        System.out.println("Hello World!");  
    }  
}
```

The diagram illustrates the structure of the code blocks. A purple box highlights the opening curly brace of the class block, with a purple arrow pointing to it from the label 'class block'. Another purple box highlights the closing curly brace of the class block, with a purple arrow pointing to it from the same 'class block' label. A green box highlights the opening curly brace of the method block, with a green arrow pointing to it from the label 'method block'. A second green box highlights the closing curly brace of the method block, with a green arrow pointing to it from the same 'method block' label.

Comments

Comments in Java help explain the code to make it easier to read. They can also be used to stop code from running, which is useful when testing new ideas.

Single-line comments start with two forward slashes (//).

Any text between // and the end of the line is ignored by Java (will not be executed).

```
// This program prints Hello World!
```

```
public class FirstProgram {  
    public static void main(String[] args) {  
        System.out.println("Hello World!");  
    }  
}
```

Comments

Multi-line comments start with `/*` and end with `*/`.

Any text between `/*` and `*/` will be ignored by the compiler.

```
/* This is a comment  
that spans multiple lines */
```

```
public class FirstProgram {  
    public static void main(String[] args) {  
        System.out.println("Hello World!");  
    }  
}
```

Conventions, Documentation, and Style

Conventions, documentation, and style play crucial roles in programming.

They ensure that code is not only functional but also maintainable and accessible to others (and you!).



Conventions

Conventions are rules on how to write code. They help everyone write code in the same way, which makes it easier to read and fix.

Easier to Understand: If everyone follows the same rules, it's quicker to understand what the code does.

Avoid Mistakes: These rules help avoid common mistakes, making the code more reliable.

Naming Conventions

Always choose descriptive and meaningful names for variables, method names, and class names.

Class names should always be written in **PascalCase**, which means that first letter of each word in the name is capitalized, and there are no spaces or underscores between words.

Documentation

Documentation means writing explanations and instructions about what the code does. This helps anyone who needs to use or fix the code.

Clear Instructions: Documentation tells you how to use the code and what each part does.

Helps New People: New team members can get up to speed quickly if the code is well-documented.

Useful for Future Fixes: Good documentation helps when you need to update or fix the code later on.

Comment your Code!!!

- Use commenting to clearly explain the purpose of the code.
- Describe what the code is intended to do, rather than how it does it.
- Avoid obvious comments.
- Comment as you go.
- Use TODOs to Indicate Incomplete Work:

```
// TODO: Replace this method with a more efficient algorithm
```

Commenting Rules for Sarah's Class

At the beginning of the programs created for an assignment, use a multi-line comment to include your name, student number, class section, date, and a summary of what the program does.

```
/**
 * Sabrina Carpenter
 * 300XXXXXX
 * COSC 111 002
 * September 4, 2024
 *
 * This program calculates the average grade for a group of students,
 * displays the highest and lowest grades, and assigns letter grades
 * based on the average.
 */

public class GradesCalculator {
    ...
}
```

Style

Style is about how the code looks. It covers things like where to put spaces, lines, and how to organize the code visually.

Easy to Read: A clean and consistent style makes the code easier to read.

Looks Professional: Well-styled code shows that the developers are careful and professional.

Better Teamwork: When everyone uses the same style, it's easier for the team to work together on the same project.

Indentation

Indentation: Indent two or four spaces (tabs are ok, too)

Spacing: Use a blank line to separate segments of the code.

```
public class SimpleProgram {  
    public static void main(String[] args) {  
        int length = 5;  
        int width = 3;  
  
        System.out.println("The area of the rectangle is " + (length * width) + ".");  
    }  
}
```

Block Styles

```
public class FirstProgram {  
    public static void main(String[] args) {  
        System.out.println("Hello World!");  
    }  
}
```

End-of-line style

```
public class FirstProgram  
{  
    public static void main(String[] args)  
    {  
        System.out.println("Hello World!");  
    }  
}
```

Next-line style

PICK ONE

Errors

Syntax Errors

Runtime Errors

Logic Errors

Syntax Errors

A syntax error, or compiler error, occurs when the code is not written according to the rules of the programming language, causing the compiler to fail.

```
public class ShowSyntaxErrors {  
    public static main(String[] args) {  
        System.out.println("Bazinga")  
    }  
}
```

Runtime Errors

A runtime error is an error that occurs during the execution of a program, which can cause the program to stop unexpectedly.

```
public class ShowRuntimeErrors {  
    public static void main(String[] args) {  
        System.out.println(1 / 0);  
    }  
}
```

Logic Errors

A logic error occurs when a program runs without crashing but produces incorrect results due to an error in the programming logic.

```
public class ShowLogicErrors {  
    public static void main(String[] args) {  
        System.out.println("1 + 2 = " );  
        System.out.println(1 + 3);  
    }  
}
```